

---

# AME 323 - Gas Dynamics, Project 1: Solving the Taylor-Maccoll equations

## Table of Contents

|                                          |   |
|------------------------------------------|---|
| .....                                    | 1 |
| Description: .....                       | 1 |
| Housekeeping: .....                      | 1 |
| Given Values: .....                      | 1 |
| Setting Up the System of Equations ..... | 2 |
| Solving the System of Equations .....    | 2 |
| Plotting .....                           | 2 |
| Functions Used .....                     | 2 |

**Authors:** Maren Kalberer, Etan Grant

**Date:** 3-27-26

## Description:

So this program is gonna do XYZ by doing XYZ

and you can reference it like TaylorMaccoll(parameter1, parameter2, etc...) No idea if this is how they want it but probably.

Test Inline LaTeX format:  $x^2 + e^{\pi i}$  Test block of LaTeX:

$$e^{\pi i} + 1 = 0$$

Bottom text lol <3

## Housekeeping:

Requires having the Aerospace Toolbox installed for certain functions

```
% Clear old variable values, outputs, and close old figures
close all;
clear;
clc;
```

## Given Values:

Known relationships, and input parameters.

```
% Atmospheric Assumptions:
R_universal = 8.314; % J/(mol K ) Universal gas constant "R"
R_air = 287; % J/(kg-degreeK) Specific gas constant for air "R_air"
```

```
gamma = 1.4; % Ratio of specific heats for standard air
rho_amb = 1.225; % Air density kg/m^3
p_amb = 1; % 1 atm? Dunno enough to know what unit we want yet I guess
T_amb = 288.15; % Std sea level tempeprature is 273.15 Kelvin?

% The given mach values for which we need to generate curves:
M_inputs = [1.5, 2.0, 5.0];

Unrecognized function or variable 'M'.

Error in TaylorMaccoll (line 204)
v_after = ((2/((gamma-1)*M^2))+1);
          ^
```

## Setting Up the System of Equations

Setmks up the functions of  $\theta$ ,  $\beta$ ,  $M$ , other crap (is this actually what we wanna do?)

## Solving the System of Equations

Solves the system and logs the results.

## Plotting

This is where the required plots are generated.

## Functions Used

Below is a set of functions used, based on lectures, NASA, and other sources

### Prantl-Meyer Angle "nu"

#### Inputs:

- Local flow mach number 'M'
- Local flow ratio of specific heats 'gamma' (usually 1.4)

#### Outputs:

- Prandtl-Meyer angle "nu", in degrees.

```
function prandtlMeyerAngle = nu(M, gamma)
    prandtlMeyerAngle = sqrt((gamma+1)/(gamma-1)) * atand(sqrt(((gamma-1)/(
(gamma+1))*(M.^2 - 1))) - atand(sqrt(M.^2 - 1));
end
```

### Oblique Shock Pressure Ratio

#### Inputs:

- Unperturbed Mach 'M' (often notated as 'M1' in diagrams)

- Local ratio of specific heats 'gamma'
- Oblique shock angle 'beta'

**Outputs:**

- Oblique-shock pressure ratio

```
function pressureRatio = obliquePR(M, gamma, beta)
    pressureRatio = 1 + ( (2*gamma)/( gamma+1 ) * ( (M^2) * (sind(beta)^2) - 1 ) );
end
```

**Oblique shock resultant Mach**

**Inputs:**

- Unperturbed flow mach number 'M' (often notated as 'M1' in diagrams)
- Oblique shock angle 'beta' (in degrees)
- Wall/flow deflection angle 'delta' (in degrees)
- Flow ratio of specific heats 'gamma,' usually 1.4

**Outputs:**

- Mach number after oblique shock & deflection, often notated as 'M2'

```
function mach = obliqueMach(M, beta, delta, gamma)
    mach = ((( gamma-1)*(M^2)*((sind(beta))^2)+2 ) / ( 2*gamma * (M^2) * ((sind(beta))^2) - (gamma-1) ) ) ^0.5 ) / ( sind(beta-delta) ) ;
end
```

**Find oblique shock angle "beta" instead of using the table.**

**Inputs:**

- Unperturbed mach "M" (AKA 'M1' in most notation),
- wall/flow deflection angle "theta" (often notated as "delta"),
- ratio of specific heats "gamma" of the flow,
- "n" specifies weak (n=0) or strong (n=1) shock case. (Usually should be weak shock case.)

**Outputs:**

- Oblique shock angle "beta" in degrees.

```
function Beta=beta(M,theta,gamma,n)
    theta=theta*pi/180; % convert to radians
    mu=asin(1/M); % Mach wave angle
    c=tan(mu)^2;
    a=( gamma-1)/2+(gamma+1)*c/2*tan(theta);
    b=( gamma+1)/2+(gamma+3)*c/2*tan(theta);
    d=sqrt(4*(1-3*a*b)^3/((27*a^2*c+9*a*b-2)^2)-1);
    Beta=atan((b+9*a*c)/(2*(1-3*a*b))-(d*(27*a^2*c+9*a*b-2)))/
```

```
(6*a*(1-3*a*b))*tan(n*pi/3+1/3*atan(1/d))*180/pi;
%{
Source:
Based on an analytical solution to the theta-beta-Mach relation given in
the following reference:  Rudd, L., and Lewis, M. J., "Comparison of
Shock Calculation Methods", AIAA Journal of Aircraft, Vol. 35, No. 4,
July-August, 1998, pp. 647-649.

Original script by Chris Plumley, undergraduate, University of Maryland.
%}
end
```

### **Mach from prandtl-meyer angle (IN DEGREES)**

#### **Inputs:**

- Ratio of specific heats "gamma" of flow, usually 1.4
- Prandtl-Meyer angle "nu" (degrees)

#### **Outputs:**

- Local mach number "mach"

```
function mach = meyerMach(gamma, nu)
    mach = flowprandtlmeyer(gamma, nu, 'nu');
end
```

### **Find delta when given M and beta (degrees)**

#### **Inputs:**

- Unperturbed Mach (AKA M1)
- oblique shock angle beta (degrees)
- Ratio of specific heats, 'gamma' of flow, usually 1.4

#### **Outputs:**

- Wall deflection angle 'delta' (in degrees)

```
function delta = deltaFinder(M, beta, gamma)
    delta = ( ( M^2 * (sind(beta)^2) ) - 1) / ( M^2 * (gamma + cosd(2*beta))
+ 2 );
    delta = 2*cotd(beta)*delta;
    delta = atand(delta);
end
```

%%Broad Steps Needed%%

```
% 1. Assume is a wedge and get M2 and shock angle Beta (have functions for
% them)
% 2. Find V' at shock angle and break into r and theta components using
% delta deflection
% 3. Want to find V' in theta direction is =0 for BC
```

```
%Velocity after Shock (v_after = v')
```

```
v_after = ((2/((gamma-1)*M^2))+1);  
function[v_prime] = v_shock(Beta,v_after,delta)  
    v_prime = v_after * cos(Beta-delta)  
end  
  
y1 = v_after;
```

*Published with MATLAB® R2024b*